MASTER VIBE-CODING PROMPT

Hotel New Era — Public Website + CMS + Complete Hotel Management System

You are a **senior software architect, UI/UX designer, full-stack engineer, database architect, hotel-management-system specialist, accounting-system specialist, and SaaS product designer**.

I want you to design and develop a complete, production-ready, scalable software platform for **Hotel New Era**.

This is NOT simply a hotel website.

The system must be designed as a **complete Hotel Management Platform**, where the public-facing Hotel New Era website is powered and controlled by a powerful backend CMS and hotel-management system.

The architecture should be designed so that the same software can later be converted into a **multi-company / multi-hotel SaaS product** and sold to other hotels.

---

1. CORE PRODUCT CONCEPT

Build one integrated platform containing:

1. Public Hotel Website
2. Hotel CMS
3. Hotel Property Management System (PMS)
4. Reservation Management
5. Front Desk
6. Guest Management / CRM
7. Room & Rate Management
8. Housekeeping Management
9. Restaurant / POS Management
10. Banquet & Conference Management
11. Event Management
12. Inventory & Store Management
13. Purchase Management
14. Supplier Management
15. Accounts Receivable

16. Accounts Payable
17. Cash & Bank Management
18. General Ledger
19. Chart of Accounts
20. Journal Entries
21. Tax Management
22. Payroll / HR integration-ready architecture
23. Fixed Assets
24. Financial Reporting
25. Hotel Operational Reporting
26. Dashboard & Business Intelligence
27. User & Role Management
28. Audit Trail
29. System Configuration
30. Notification Management
31. Document/File Management
32. API Integration Layer
33. Multi-company / multi-property architecture

The public website and internal management system must operate from the same centralized database and backend architecture.

---

2. IMPORTANT ARCHITECTURAL PRINCIPLE

Do NOT build the website first and then bolt a CMS onto it.

Instead, build the platform around the following hierarchy:

PUBLIC WEBSITE
↓
CMS
↓
HOTEL APPLICATION LAYER
↓
PMS / POS / CRM / INVENTORY / ACCOUNTS / REPORTING

↓
POSTGRESQL DATABASE

The public website should obtain dynamic content from the CMS.

For example:

- Rooms displayed on website → controlled from Room Management
- Room photographs → controlled through CMS
- Room descriptions → CMS
- Room facilities → CMS
- Room rates → Rate Management
- Availability → Reservation/PMS engine
- Offers → CMS + Rate/Promotion engine
- Restaurant information → CMS
- Events → Event/Banquet module
- Contact information → CMS
- Hotel gallery → CMS
- Testimonials → CMS
- Blog/news → CMS
- SEO metadata → CMS
- Homepage sections → CMS
- Website banners → CMS
- Booking engine → PMS/Reservation system

A CMS administrator should be able to manage the website without modifying code.

---

3. TECHNOLOGY REQUIREMENTS

Use a modern, maintainable and scalable technology stack.

Database

Use:

PostgreSQL

Design a properly normalized relational database.

Do not use mock data as the permanent architecture.

Use:

- Primary keys
- Foreign keys
- Unique constraints
- Check constraints
- Indexes
- Proper transactions
- Audit fields
- Created date/time
- Updated date/time
- Created by
- Updated by
- Soft deletion where appropriate
- Database-level integrity

The database must be designed for future multi-company and multi-hotel operation.

---

4. RECOMMENDED APPLICATION ARCHITECTURE

Use a modern full-stack architecture.

Prefer:

- TypeScript
- React / Next.js or an equally strong modern frontend framework
- Node.js backend or another suitable strongly typed backend
- PostgreSQL
- REST API and/or well-structured API layer
- ORM such as Prisma or equivalent
- Modular architecture
- Responsive design
- Component-based UI

- Secure authentication
- Role-based authorization

If you select an alternative technology, explain why before implementation.

Do not introduce unnecessary technologies.

The system must be easy for another professional developer to maintain later.

---

5. MULTI-TENANT / MULTI-HOTEL READY ARCHITECTURE

Although the first implementation is for Hotel New Era, design the database and application so it can eventually support:

Company
→ Hotel / Property
→ Branch
→ Departments
→ Users
→ Customers / Guests
→ Transactions

A future customer should be able to create:

Company A
├— Hotel A
├— Hotel B
└— Hotel C

while another customer can have:

Company B
└— Hotel D

Data must remain properly isolated.

Do not hard-code Hotel New Era throughout the application.

Use configuration and tenant/property relationships.

---

6. PUBLIC HOTEL WEBSITE

Create a premium, modern, elegant hotel website.

The website should include:

Homepage

Sections:

- Hero banner
- Hotel introduction
- Booking/search widget
- Rooms
- Special offers
- Dining
- Facilities
- Events / banquet
- Gallery
- Experiences
- About Hotel New Era
- Location
- Testimonials
- Latest news/blog
- Contact
- Footer

The homepage must be CMS-driven.

The administrator should be able to:

- Change hero image
- Change hero text
- Change buttons
- Rearrange sections
- Enable/disable sections
- Edit headings
- Edit descriptions
- Upload images
- Manage promotional banners
- Manage offers
- Manage testimonials

- Manage blog posts

without coding.

---

7. WEBSITE PAGES

Create dynamic pages for:

- Home
- About Us
- Rooms
- Room Details
- Dining
- Restaurants
- Facilities
- Events
- Banquets
- Conference
- Gallery
- Offers
- Packages
- Experiences
- Blog / News
- Contact
- Location
- Privacy Policy
- Terms & Conditions
- Booking Terms
- Cancellation Policy
- Careers

The CMS should allow administrators to create additional pages.

---

8. HOTEL BOOKING ENGINE

The website must include an integrated booking engine.

Guest should be able to enter:

- Check-in date
- Check-out date
- Number of adults
- Number of children
- Number of rooms
- Room type

System should display:

- Available rooms
- Occupancy
- Rate
- Taxes
- Discounts
- Packages
- Promotions
- Total amount

Guest should be able to:

- Select room
- Enter guest details
- Add services
- Review booking
- Make payment if payment gateway is connected
- Receive booking confirmation

Booking should automatically create a reservation inside the PMS.

---

9. PMS — PROPERTY MANAGEMENT SYSTEM

Create a complete PMS.

Main modules:

Reservation

- New reservation
- Modify reservation
- Cancel reservation
- Group reservation
- Corporate reservation
- Walk-in reservation
- Online reservation
- OTA reservation
- Reservation source
- Rate plan
- Package
- Deposit
- Advance payment
- Cancellation policy
- No-show
- Confirmation
- Waitlist

Front Desk

- Arrival
- Check-in
- In-house guests
- Room transfer
- Guest folio
- Additional charges
- Check-out
- Early check-in
- Late check-out

- Guest history

Room Management

- Room types
- Individual rooms
- Floor
- Room status
- Occupancy
- Amenities
- Room features
- Room maintenance status
- Out of order
- Out of service
- Housekeeping status

Room status examples:

- Vacant Clean
- Vacant Dirty
- Occupied
- Reserved
- Due Out
- Out of Order
- Out of Service
- Inspection Required

---

10. HOUSEKEEPING

Create housekeeping management.

Features:

- Daily room status
- Cleaning assignments
- Housekeeper assignment

- Cleaning checklist
- Room inspection
- Maintenance request
- Lost & found
- Linen management
- Housekeeping supplies
- Room discrepancy reporting

Dashboard should show:

Clean
Dirty
Inspected
Occupied
Vacant
Out of Order

---

11. GUEST CRM

Create a centralized guest database.

Guest profile should contain:

- Name
- Contact
- Email
- Address
- Country
- ID/passport details where legally appropriate
- Preferences
- Stay history
- Spending history
- Complaints
- Feedback
- Loyalty information

- Corporate relationship
- Notes

The same guest should not unnecessarily have multiple duplicate profiles.

Implement duplicate detection where practical.

---

12. RESTAURANT / POS

Design an integrated restaurant/POS module.

Features:

- Restaurants
- Outlets
- Tables
- Menu items
- Categories
- Modifiers
- Kitchen orders
- KOT
- Billing
- Discounts
- Taxes
- Service charges
- Payment methods
- Split bills
- Guest room posting
- Cashier closing
- Restaurant sales reports

Important:

A restaurant charge should be capable of being posted directly to a guest's hotel folio.

Example:

Guest Room 205
→ Restaurant bill
→ Post to room
→ Guest checks out
→ Hotel folio includes room + restaurant + other charges

---

13. BANQUET / EVENTS / CONFERENCE

Create a complete banquet management module.

Features:

- Halls
- Hall capacity
- Hall configuration
- Event calendar
- Event booking
- Corporate events
- Weddings
- Conferences
- Seminars
- Meetings
- Packages
- Menu packages
- Decoration
- Equipment
- AV equipment
- Catering
- Event quotation
- Event contract
- Advance payment
- Final billing
- Event folio

The system should prevent double-booking of halls.

---

14. INVENTORY & STORES

Create inventory management.

Modules:

- Items
- Categories
- Units
- Warehouses
- Stores
- Opening stock
- Purchase
- Purchase return
- Stock transfer
- Stock adjustment
- Stock issue
- Consumption
- Wastage
- Physical stock count
- Reorder level
- Batch tracking where necessary

Support multiple stores such as:

- Main Store
- Kitchen Store
- Bar Store
- Housekeeping Store
- Maintenance Store
- Engineering Store

---

15. PROCUREMENT

Create:

- Supplier master
- Purchase requisition
- Purchase order
- Goods receipt
- Purchase invoice
- Purchase return
- Supplier payment
- Supplier statement

Procurement should integrate with inventory and accounting.

---

16. ACCOUNTING / FINANCE

This is extremely important.

The system should ultimately function as a complete accounting system.

Create:

Chart of Accounts

Support:

- Assets
- Liabilities
- Equity
- Revenue
- Cost of Sales
- Expenses
- Other Income
- Other Expenses

Allow configurable account groups and subgroups.

General Ledger

Features:

- Journal entries
- Debit
- Credit
- Narration
- Voucher number
- Posting date
- Reference
- Cost center
- Department
- Project
- User
- Source module

Every financial transaction should have a source.

For example:

Room sale
→ Revenue
→ Accounts receivable/cash
→ General ledger

Restaurant sale
→ Restaurant revenue
→ Tax
→ Cash/receivable

Purchase
→ Inventory/expense
→ Accounts payable

Payment
→ Cash/bank
→ Receivable/payable

---

17. ACCOUNTING INTEGRATION

Do NOT make accounting a completely separate system.

Every operational module should be capable of generating accounting entries.

Examples:

Reservation advance
→ Dr Cash/Bank
→ Cr Guest Advance

Room sale
→ Dr Guest Receivable
→ Cr Room Revenue

Restaurant sale
→ Dr Cash/Guest Receivable
→ Cr Food Revenue
→ Cr Beverage Revenue
→ Cr Tax Payable

Purchase
→ Dr Inventory/Expense
→ Dr Input Tax
→ Cr Supplier Payable

Supplier payment
→ Dr Supplier Payable
→ Cr Bank/Cash

The exact accounting rules must be configurable.

---

18. TAX SYSTEM

Design a configurable tax engine.

It should support:

- Tax types
- Tax rates
- Inclusive tax
- Exclusive tax
- Taxable items
- Tax exemptions
- Input tax
- Output tax
- Tax reports

- Tax-period configuration

Do not hard-code tax rates into the application.

The tax configuration should be manageable from the admin panel.

---

19. FINANCIAL REPORTING

Create:

- Trial Balance
- General Ledger
- Cash Book
- Bank Book
- Profit & Loss
- Balance Sheet
- Cash Flow Statement
- Accounts Receivable
- Accounts Payable
- Supplier Statement
- Customer Statement
- Revenue Report
- Expense Report
- Tax Report
- Department-wise P&L
- Cost-center reports
- Daily Revenue Report
- Night Audit Report

Reports should support:

- Date range
- Property
- Department
- Account

- Cost center
- User
- Export to Excel
- Export to PDF
- Print

---

20. NIGHT AUDIT

Implement a hotel night-audit workflow.

The system should verify:

- Arrivals
- Departures
- No-shows
- Room charges
- Restaurant charges
- Other revenue
- Taxes
- Discounts
- Payments
- Guest balances
- Cashier balances

Then allow authorized users to close the business day.

Night audit should be controlled and fully logged.

---

21. DASHBOARD

Create separate dashboards.

General Management Dashboard

Show:

- Occupancy
- ADR

- RevPAR
- Room revenue
- Restaurant revenue
- Total revenue
- Expenses
- Profit
- Arrivals
- Departures
- In-house guests
- Pending payments
- Outstanding receivables
- Outstanding payables

Front Desk Dashboard

Show:

- Today's arrivals
- Today's departures
- In-house guests
- Vacant rooms
- Dirty rooms
- Available rooms
- Reservations
- VIP guests

Finance Dashboard

Show:

- Revenue
- Expenses
- Cash
- Bank
- Receivables

- Payables
- Profit
- Tax liability

Housekeeping Dashboard

Show:

- Rooms to clean
- Rooms cleaned
- Rooms inspected
- Maintenance issues

---

22. CMS

The CMS must be a serious enterprise CMS.

Create:

Content Management

- Pages
- Sections
- Menus
- Navigation
- Banners
- Images
- Videos
- Gallery
- Blog
- News
- Testimonials
- Offers
- Packages
- FAQs

SEO

Every page should support:

- SEO title
- Meta description
- Keywords
- Canonical URL
- Open Graph data
- Social sharing image
- URL slug

Generate sitemap functionality.

---

23. MEDIA LIBRARY

Create a media library.

Features:

- Upload image
- Upload multiple images
- Image preview
- Categories
- Search
- Tags
- Alt text
- File metadata
- Delete
- Replace
- Reuse existing images

Do not unnecessarily duplicate uploaded files.

---

24. USER MANAGEMENT

Create RBAC.

Roles could include:

- Super Administrator
- Company Administrator
- General Manager
- Front Office Manager
- Front Desk User
- Reservation User
- Accountant
- Finance Manager
- Cashier
- Restaurant Manager
- Restaurant Cashier
- Housekeeping Manager
- Housekeeper
- Store Manager
- Purchase Officer
- HR Manager
- Event Manager
- Auditor
- Report Viewer

Permissions must be granular.

Example:

User can:

View Reservations
Create Reservation
Edit Reservation
Cancel Reservation
View Financial Data
Create Journal
Post Journal
Approve Journal
View Payroll

Edit CMS
Publish Website Content

---

25. APPROVAL WORKFLOW

Implement configurable approval workflows.

Examples:

Purchase Requisition
→ Department Manager
→ Purchase Manager
→ Finance
→ Approval

Journal Entry
→ Prepared
→ Submitted
→ Reviewed
→ Approved
→ Posted

Purchase Order
→ Prepared
→ Approved

Use statuses rather than simply allowing every user to modify everything.

---

26. AUDIT TRAIL

Every important action must be recorded.

Record:

- User
- Date/time
- IP/device where appropriate
- Module
- Record
- Action
- Old value
- New value

Examples:

Who changed room rate?

Who cancelled reservation?

Who changed accounting entry?

Who approved purchase?

Who modified website content?

Who deleted a record?

Audit logs should not be casually editable or deletable.

---

27. UI/UX DESIGN

The visual identity should be inspired by the attached OrangeHRM reference image.

Do NOT copy the OrangeHRM interface.

Use the image only as a **color and visual-style reference**.

Desired visual language

- Light overall background
- White cards
- Orange primary accent
- Golden/yellow secondary accent
- Fresh green secondary accent
- Dark charcoal text
- Very light grey borders
- Subtle shadows
- Rounded corners
- Clean typography
- Spacious layout
- Professional enterprise SaaS appearance
- Minimal visual clutter

Suggested design palette:

Primary Orange:
#F28C00

Deep Orange:
#E87500

Golden:
#F4B400

Light Golden:
#FFF3D6

Green:
#43A047

Light Green:
#EAF6EA

Background:
#F7F8F9

Card:
#FFFFFF

Text:
#263238

Secondary Text:
#6B7280

Border:
#E5E7EB

These colors are starting points. Refine them where necessary to create a premium visual system.

The website should feel like:

Modern Hotel + Professional Enterprise Software + Warm Hospitality

not like generic accounting software.

---

28. LOGIN PAGE

Create an elegant login page inspired by the visual character of the attached reference.

Use:

- Light background
- White login card
- Orange/gold accents

- Green highlights
- Hotel New Era branding
- Logo
- Username/email
- Password
- Remember me
- Forgot password
- Login
- Optional 2FA

The login page should feel premium and professional.

---

29. ADMIN APPLICATION LAYOUT

Use a modern SaaS dashboard.

Desktop:

Left sidebar
→ Module navigation

Top bar
→ Search
→ Notifications
→ Current property
→ User profile

Main content
→ Dashboard / module

Sidebar should support collapsing.

Mobile:

Use responsive navigation.

---

30. DATABASE DESIGN

Before coding major features, create a complete database architecture.

At minimum consider entities for:

- companies

- properties
- branches
- users
- roles
- permissions
- guests
- guest_documents
- guest_preferences
- room_types
- rooms
- room_features
- room_status
- reservations
- reservation_rooms
- rate_plans
- rates
- promotions
- packages
- folios
- folio_items
- payments
- payment_methods
- restaurants
- restaurant_tables
- menu_categories
- menu_items
- modifiers
- kitchen_orders
- kitchen_order_items

- banquet_halls
- events
- event_packages
- inventory_items
- warehouses
- stock_transactions
- suppliers
- purchase_requisitions
- purchase_orders
- goods_receipts
- purchase_invoices
- accounts
- journal_entries
- journal_entry_lines
- taxes
- cost_centers
- departments
- fixed_assets
- employees
- website_pages
- website_sections
- website_menus
- media
- blog_posts
- offers
- testimonials
- notifications
- audit_logs
- system_settings

This is not an exhaustive list.

Normalize the database properly.

Avoid creating unnecessary tables.

Use appropriate indexes and relationships.

---

31. API DESIGN

Create clean APIs between frontend and backend.

Organize APIs by domain:

/api/auth
/api/users
/api/companies
/api/properties
/api/rooms
/api/reservations
/api/guests
/api/frontdesk
/api/housekeeping
/api/restaurant
/api/banquet
/api/inventory
/api/purchases
/api/accounting
/api/reports
/api/cms
/api/media
/api/settings

Use consistent error handling.

Validate all inputs.

Never trust frontend validation alone.

---

32. SECURITY

Implement:

- Password hashing
- Secure authentication
- Role-based authorization

- Session management
- CSRF protection where applicable
- Input validation
- SQL injection protection
- XSS protection
- Rate limiting
- Secure file uploads
- API authorization
- Audit logs
- Environment variables for secrets

Never expose database credentials in frontend code.

Never hard-code API secrets.

---

33. PERFORMANCE

The system should be designed for future growth.

Use:

- Database indexing
- Pagination
- Lazy loading
- Caching where appropriate
- Optimized queries
- Server-side rendering where beneficial
- Image optimization
- Background processing where required

Do not load thousands of records unnecessarily.

---

34. RESPONSIVE DESIGN

The public website must work perfectly on:

- Desktop

- Laptop
- Tablet
- Mobile

The admin system should prioritize desktop but remain usable on tablets.

---

35. HOTEL WEBSITE BRANDING

Use the brand:

Hotel New Era

The design should communicate:

- Hospitality
- Trust
- Comfort
- Heritage
- Professionalism
- Premium service
- Nepalese character

Avoid overly flashy luxury-hotel design.

The site should be elegant, warm and modern.

---

36. CONTENT ARCHITECTURE

Do not hard-code website content.

For example, instead of:

"Welcome to Hotel New Era"

hard-coded into frontend components, retrieve the content from CMS.

Use structured content such as:

Homepage Hero
Title
Subtitle
Description
CTA

Image
Status
Sort Order

This will allow the same software to later power other hotels.

---

37. MULTI-LANGUAGE READY

Design the database and CMS to support multiple languages.

Initially support:

- English
- Nepali

Do not hard-code language-specific content into components.

Use translation structures.

---

38. NEPALI DATE / CALENDAR READY

The architecture should be capable of supporting:

- Gregorian date
- Nepali Bikram Sambat date

The database should preferably store standardized Gregorian timestamps while the UI/reporting layer can display Nepali dates where required.

Do not corrupt database date calculations by storing Bikram Sambat as the primary database date.

---

39. NOTIFICATION SYSTEM

Create notification architecture for:

- Reservation confirmation
- Booking modification
- Cancellation
- Payment
- Check-in
- Check-out
- Low inventory

- Approval requests
- Maintenance requests
- System notifications

Make the notification engine extensible for:

- Email
- SMS
- WhatsApp
- In-app notifications

---

40. INTEGRATION READY

Design the system so future integrations can be added for:

- Payment gateways
- OTA/channel managers
- Online travel agencies
- SMS gateways
- Email
- WhatsApp
- Accounting systems
- Biometric attendance
- E-invoicing
- Tax systems
- Google services
- Maps
- Analytics
- POS hardware
- Printers

Do not implement fake integrations.

Create clean interfaces/services so integrations can be plugged in later.

---

41. REPORT EXPORT

Reports should support:

- PDF
- Excel
- CSV
- Print

Use professional report layouts.

Reports should display:

Hotel name
Property
Report title
Date range
Generated by
Generated date
Relevant filters
Data
Totals
Footer

---

42. DASHBOARD DESIGN PRINCIPLES

Avoid creating dashboards full of meaningless charts.

Every dashboard component must answer a business question.

Examples:

"How many rooms are occupied?"

"How much revenue was generated today?"

"What is outstanding from guests?"

"What are today's arrivals?"

"Which rooms require housekeeping?"

"What are today's restaurant sales?"

"What expenses were incurred?"

"What is the hotel's current profitability?"

Use:

- KPI cards
- Tables
- Charts
- Status indicators
- Trend indicators
- Action buttons

---

43. WORKFLOW DEVELOPMENT METHOD

DO NOT attempt to generate the entire application in one giant code dump.

Work in phases.

Phase 1 — Architecture

First produce:

- System architecture
- Technology stack
- Folder structure
- Database ERD
- Database schema
- Module dependency map
- Authentication architecture
- Tenant architecture
- API architecture

Do not start coding until the architecture is internally consistent.

Phase 2 — Design System

Create:

- Color system
- Typography
- Buttons
- Forms
- Tables

- Cards
- Modals
- Navigation
- Alerts
- Status badges
- Dashboard components
- Login page

Phase 3 — Core Infrastructure

Build:

- Database
- Authentication
- Authorization
- User management
- Company/property setup
- Audit logging
- Settings

Phase 4 — PMS

Build:

- Room management
- Room types
- Reservations
- Guests
- Front desk
- Check-in
- Check-out
- Folios

Phase 5 — Operations

Build:

- Housekeeping

- Restaurant/POS
- Banquet
- Events
- Inventory
- Procurement

Phase 6 — Accounting

Build:

- Chart of accounts
- Journal
- Ledger
- Receivables
- Payables
- Cash/bank
- Tax
- Financial reports

Phase 7 — CMS

Build:

- Pages
- Homepage builder
- Rooms
- Offers
- Gallery
- Blog
- Events
- SEO
- Media library

Phase 8 — Public Website

Connect the CMS and PMS to the public website.

Phase 9 — Reporting

Build management and financial reports.

Phase 10 — Testing & Security

Perform:

- Functional testing
- Database integrity testing
- Permission testing
- Security testing
- Responsive testing
- Performance testing
- Workflow testing

---

44. CRITICAL VIBE-CODING RULE

Do not make assumptions silently.

Whenever a requirement is ambiguous:

1. Identify the ambiguity.
2. Recommend the best professional approach.
3. Explain the implication briefly.
4. Then proceed using the recommended approach unless it requires a major business decision.

Do not repeatedly ask unnecessary questions.

---

45. CODE QUALITY

Write production-quality code.

Avoid:

- Giant components
- Duplicate code
- Hard-coded business logic
- Hard-coded hotel data
- Hard-coded tax rates

- Hard-coded permissions
- Fake database calls
- Fake API responses
- Temporary mock data presented as production functionality
- Unnecessary dependencies

Use:

- Reusable components
- Services
- Controllers/routes
- Validation schemas
- Type safety
- Database transactions
- Clear naming
- Modular architecture
- Error handling

---

46. BUSINESS LOGIC RULE

Separate:

UI
→ API
→ Business Logic
→ Database

Do not put critical accounting or hotel business logic directly into frontend components.

For example:

Room checkout
should trigger a backend transaction.

The backend should:

1. Validate reservation
2. Calculate folio
3. Calculate taxes

4. Validate payment
5. Update room status
6. Create accounting entries where applicable
7. Record audit trail
8. Commit database transaction

Everything should succeed or fail atomically where appropriate.

---

47. INITIAL ADMIN SETUP

Create a setup wizard.

First installation should allow:

Company Name
Property Name
Address
Contact
Email
Currency
Timezone
Fiscal Year
Date Format
Language
Tax Configuration
Admin User

Then:

Create Room Types
Create Rooms
Create Rate Plans
Create Accounts
Create Departments
Create Stores
Create Restaurants
Create Banquet Halls

---

48. DEMO DATA

Create realistic demo data ONLY for development/testing.

Clearly separate demo/seed data from production data.

Do not allow fake demo transactions to contaminate the actual accounting system.

---

49. ERROR HANDLING

Create professional error states.

Examples:

- No data
- Loading
- Permission denied
- Invalid input
- Duplicate record
- Database error
- Network error
- Session expired

Never display raw database errors to users.

---

50. FINAL PRODUCT OBJECTIVE

The final product should feel like a combination of:

Modern Hotel Website
+
Hotel PMS
+
Restaurant POS
+
Banquet Management
+
Inventory
+
Procurement
+
Accounting ERP
+
CRM
+
CMS

+

Management Dashboard

but all integrated into one coherent platform.

The public website should be beautiful enough for hotel guests.

The backend should be powerful enough for accountants, hotel managers and operational staff.

The accounting architecture must be robust enough for professional financial reporting.

The database must be strong enough to eventually support multiple hotels and multiple companies.

---

51. STARTING INSTRUCTION

Before writing substantial application code, do the following:

Step 1

Analyze the entire requirement.

Step 2

Identify missing architectural decisions.

Step 3

Propose the technology stack.

Step 4

Create the complete module hierarchy.

Step 5

Create the database entity relationship architecture.

Step 6

Create the initial PostgreSQL schema.

Step 7

Create the application folder structure.

Step 8

Create the design system based on the attached OrangeHRM-inspired color direction.

Step 9

Create the authentication and multi-tenant architecture.

Step 10

Build the application incrementally.

At the end of each phase:

- Verify that the code runs.
- Check database migrations.
- Check relationships.
- Check permissions.
- Check responsive behavior.
- Fix errors before moving to the next phase.

Do not sacrifice architecture for speed.

Do not create a visually impressive prototype that cannot become a real production system.

The goal is to build a **real, scalable Hotel Management SaaS platform**, beginning with Hotel New Era as the first property.

When a feature is implemented, connect it to the actual PostgreSQL database and real application logic rather than leaving it as a frontend mock.

Start by producing the **System Architecture + Module Map + Database ERD + Technology Stack + Development Roadmap**. Do not begin with the homepage.